Section 3.4: Backpropagation

Computing Gradients with the Chain Rule

AI-IE Class — Lecture Notes

---

1 The Goal

From Section 3.2 (Gradient Descent), we know that to update a weight (and bias) w we need its gradient:

$$w \longleftarrow w - \eta \frac{\partial J}{\partial w}.$$

From Section 3.3 (Forward Pass) we computed a prediction $\hat{y} = 0.637$ and a loss $J = 0.132$ for the input $(x_1, x_2) = (0, 1)$.

Now we need $\partial J/\partial w$ for every weight in the network. The difficulty is that the loss depends on a weight only *indirectly*: $w \to z \to h \to \cdots \to \hat{y} \to J$. Each arrow is a function, and we need to differentiate through all of them. The tool that makes this systematic is the **chain rule**.

2 The Chain Rule

If y depends on u, and u depends on x, then:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}.$$

This extends to any chain of functions. For three nested functions $y = f(g(h(x)))$, let $v = h(x)$ and $u = g(v)$:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx}.$$

Example 2.1. Let $y = (2x + 1)^2$. Write $u = 2x + 1$ so that $y = u^2$:

$$\frac{dy}{dx} = \underbrace{\frac{dy}{du}}_{2u} \cdot \underbrace{\frac{du}{dx}}_{2} = 2(2x + 1) \cdot 2 = 4(2x + 1).$$

Backpropagation is simply the chain rule applied *backwards* through the network — one layer at a time. We start from $\partial J/\partial \hat{y}$ (easy to compute) and work leftward, accumulating the product at each layer.

3 What We Know from Section 3.3

The network has the following (current) parameters:

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ 0.5 & 0.5 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.5 \\ 0.0 \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} 0.5 & 0.5 \end{bmatrix}, \quad b^{(2)} = 0.0.$$

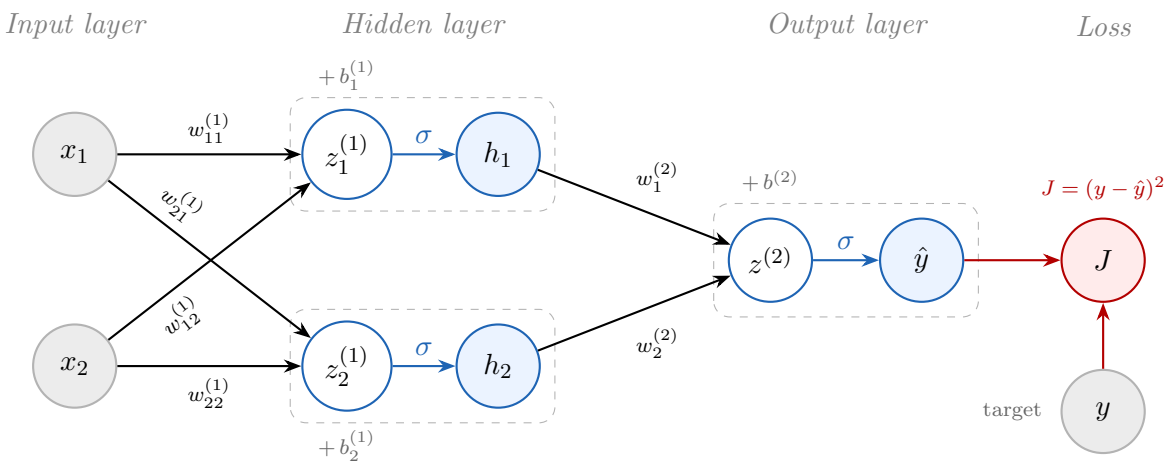


Figure 1: The complete forward pass.

Quantity	Symbol	Value
Input	$(x_1,\ x_2)$	$(0,\ 1)$
Target	y	1
Hidden pre-activation 1	$z_1^{(1)}$	0.0
Hidden pre-activation 2	$z_2^{(1)}$	0.5
Hidden activation 1	h_1	0.500
Hidden activation 2	h_2	0.622
Output pre-activation	$z^{(2)}$	0.561
Prediction	$\hat{y}$	0.637
Loss	J	0.132

We also need the sigmoid derivatives $\sigma'(z) = \sigma(z)(1-\sigma(z))$ at the values we computed. We can use Python to evaluate them:

```
def sigmoid_p(x): # derivative of sigmoid
s = sigmoid(x)
return s * (1 - s)
```

4 Backward Pass: Output Layer

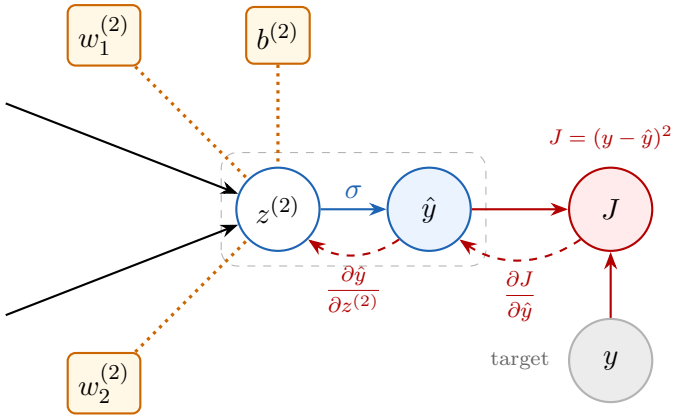


Figure 2: Zooming in on the output neuron. The dotted lines show that $w_1^{(2)}$, $w_2^{(2)}$ and $b^{(2)}$ affect the loss only through $z^{(2)}$, so each gradient is the chain $J \to \hat{y} \to z^{(2)} \to$ parameter.

We work right to left through the computation graph:

$$J \xleftarrow{\partial J/\partial \hat{y}} \hat{y} \xleftarrow{\partial \hat{y}/\partial z^{(2)}} z^{(2)} \xleftarrow{\partial z^{(2)}/\partial w_j^{(2)}} w_j^{(2)} \qquad (j = 1, 2)$$

$$J \xleftarrow{\partial J/\partial \hat{y}} \hat{y} \xleftarrow{\partial \hat{y}/\partial z^{(2)}} z^{(2)} \xleftarrow{\partial z^{(2)}/\partial b^{(2)}} b^{(2)}$$

The output neuron has three parameters: two weights $w_1^{(2)}, w_2^{(2)}$ and one bias $b^{(2)}$. All three reach J through $z^{(2)}$ (see Figure 2), so their chains share the first two links and differ only in the last.

Step 1 Gradient of the loss with respect to $\hat{y}$

$$J = (y - \hat{y})^2 \implies \frac{\partial J}{\partial \hat{y}} = 2(\hat{y} - y) = 2(0.637 - 1) = -0.726$$

The negative sign means: increasing $\hat{y}$ would *decrease* J. Since $\hat{y} < y$, the network is predicting too low.

Step 2 Gradient through the output sigmoid

$$\hat{y} = \sigma(z^{(2)}) \implies \frac{\partial \hat{y}}{\partial z^{(2)}} = \sigma'(z^{(2)}) = \hat{y}(1 - \hat{y}) = 0.637 \times 0.363 \approx 0.231$$

By the chain rule, the gradient arriving at $z^{(2)}$ is:

$$\frac{\partial J}{\partial z^{(2)}} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(2)}} = (-0.726)(0.231) \approx -0.168$$

We will call this quantity $\delta^{(2)} = -0.168$. It is the **error signal** at the output neuron: how much the loss changes per unit change in $z^{(2)}$.

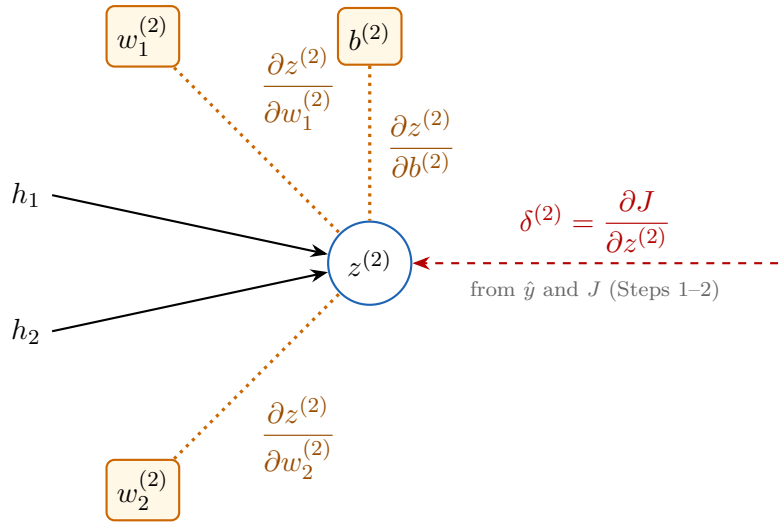


Figure 3: After Steps 1–2, everything to the right of $z^{(2)}$ collapses into the error signal $\delta^{(2)}$. Each parameter's gradient is $\delta^{(2)}$ times the label on its link.

Step 3 Gradients for the output weights

Because $z^{(2)} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + b^{(2)}$, we have:

$$\frac{\partial z^{(2)}}{\partial w_1^{(2)}} = h_1, \qquad \frac{\partial z^{(2)}}{\partial w_2^{(2)}} = h_2, \qquad \frac{\partial z^{(2)}}{\partial b^{(2)}} = 1.$$

Applying the chain rule one more time:

$$\frac{\partial J}{\partial w_1^{(2)}} = \delta^{(2)} \cdot h_1 = (-0.168)(0.500) = -0.084$$

$$\frac{\partial J}{\partial w_2^{(2)}} = \delta^{(2)} \cdot h_2 = (-0.168)(0.622) \approx -0.105$$

$$\frac{\partial J}{\partial b^{(2)}} = \delta^{(2)} \cdot 1 = -0.168$$

Pattern observed. For any output weight $w_i^{(2)}$, the gradient is:

$$\frac{\partial J}{\partial w_i^{(2)}} = \delta^{(2)} \cdot h_i \qquad \text{(error signal} \times \text{incoming activation)}.$$

This pattern repeats at every layer.

5 Backward Pass: Hidden Layer

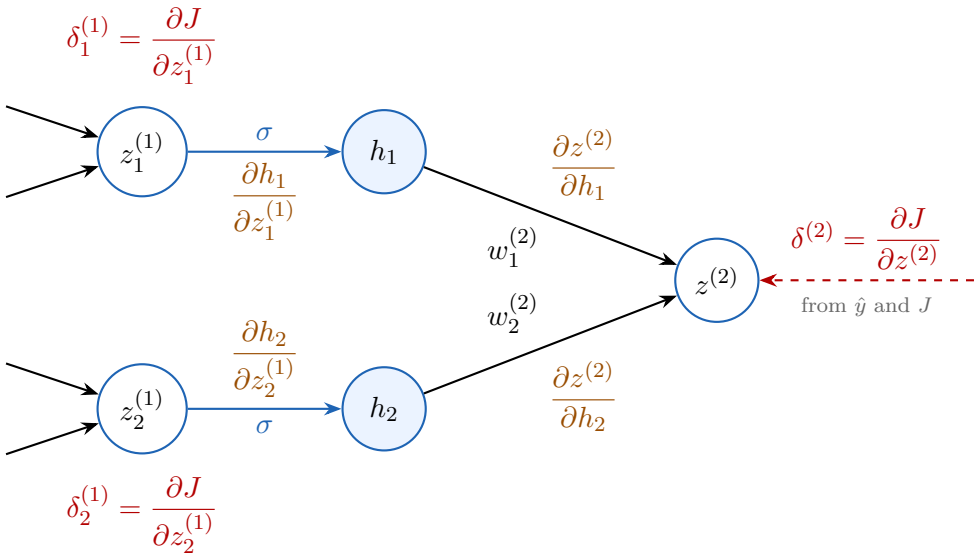


Figure 4: The error signal $\delta^{(2)}$ flows backwards through each hidden unit. Multiplying the labels along a path gives $\delta_i^{(1)} = \delta^{(2)} \cdot \frac{\partial z^{(2)}}{\partial h_i} \cdot \frac{\partial h_i}{\partial z_i^{(1)}}$.

To update $\mathbf{W}^{(1)}$ and $\mathbf{b}^{(1)}$ we need $\partial J/\partial w_{ij}^{(1)}$ and $\partial J/\partial b_i^{(1)}$. Working right to left, the chains are:

$$J \xleftarrow{\partial J/\partial \hat{y}} \hat{y} \xleftarrow{\partial \hat{y}/\partial z^{(2)}} z^{(2)} \xleftarrow{\partial z^{(2)}/\partial h_i} h_i \xleftarrow{\partial h_i/\partial z_i^{(1)}} z_i^{(1)} \xleftarrow{\partial z_i^{(1)}/\partial w_{ij}^{(1)}} w_{ij}^{(1)}$$

$$J \xleftarrow{\partial J/\partial \hat{y}} \hat{y} \xleftarrow{\partial \hat{y}/\partial z^{(2)}} z^{(2)} \xleftarrow{\partial z^{(2)}/\partial h_i} h_i \xleftarrow{\partial h_i/\partial z_i^{(1)}} z_i^{(1)} \xleftarrow{\partial z_i^{(1)}/\partial b_i^{(1)}} b_i^{(1)}$$

As in the output layer, the weight and bias chains share every link except the last.

We already know $\delta^{(2)} = \partial J/\partial z^{(2)} = -0.168$. The chain rule gives the error signal at hidden unit i:

Error signal at hidden unit i:

$$\delta_i^{(1)} = \frac{\partial J}{\partial z_i^{(1)}} = \delta^{(2)} \cdot w_i^{(2)} \cdot \sigma'\left(z_i^{(1)}\right).$$

Three factors multiplied together:

1. $\delta^{(2)}$: error signal from the layer ahead.
2. $w_i^{(2)}$: how strongly h_i connects to the output.
3. $\sigma'(z_i^{(1)})$: how sensitive h_i is to its own pre-activation.

Once $\delta_i^{(1)}$ is known, the weight gradients follow the same rule as before:

$$\frac{\partial J}{\partial w_{ij}^{(1)}} = \delta_i^{(1)} \cdot x_j, \qquad \frac{\partial J}{\partial b_i^{(1)}} = \delta_i^{(1)}.$$

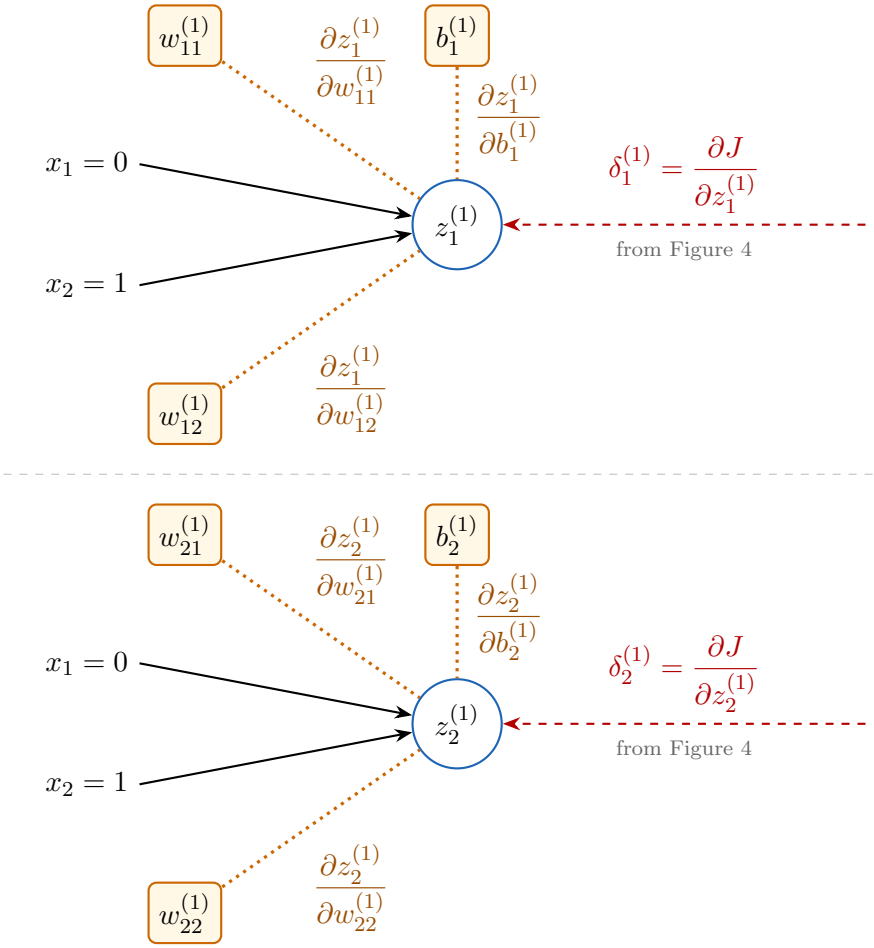


Figure 5: Zooming in on the hidden pre-activations. The hidden layer has four weights and two biases, each linked to one $z_i^{(1)}$. As in the output layer, each parameter's gradient is $\delta_i^{(1)}$ times the label on its link.

5.1 Applying the Pattern to h_1

$$\delta_1^{(1)} = \delta^{(2)} \cdot w_1^{(2)} \cdot \sigma'(z_1^{(1)}) = (-0.168)(0.5)(0.250) = -0.021$$

Because $z_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}$, the local derivatives are:

$$\frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}} = x_1 = 0, \qquad \frac{\partial z_1^{(1)}}{\partial w_{12}^{(1)}} = x_2 = 1, \qquad \frac{\partial z_1^{(1)}}{\partial b_1^{(1)}} = 1.$$

The gradients for row 1 of $\mathbf{W}^{(1)}$ and for $b_1^{(1)}$:

$$\frac{\partial J}{\partial w_{11}^{(1)}} = \delta_1^{(1)} \cdot x_1 = (-0.021)(0) = 0.000$$
$$\frac{\partial J}{\partial w_{12}^{(1)}} = \delta_1^{(1)} \cdot x_2 = (-0.021)(1) = -0.021$$
$$\frac{\partial J}{\partial b_1^{(1)}} = \delta_1^{(1)} \cdot 1 = -0.021$$

$\partial J / \partial w_{11}^{(1)} = 0$ because $x_1 = 0$: this input was zero, so the weight connecting it had no effect on the prediction — and therefore no effect on the loss.

5.2 Applying the Pattern to h_2

$$\delta_2^{(1)} = \delta^{(2)} \cdot w_2^{(2)} \cdot \sigma'(z_2^{(1)}) = (-0.168)(0.5)(0.235) \approx -0.020$$

Because $z_2^{(1)} = w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)}$, the local derivatives are:

$$\frac{\partial z_2^{(1)}}{\partial w_{21}^{(1)}} = x_1 = 0, \qquad \frac{\partial z_2^{(1)}}{\partial w_{22}^{(1)}} = x_2 = 1, \qquad \frac{\partial z_2^{(1)}}{\partial b_2^{(1)}} = 1.$$

The gradients for row 2 of $\mathbf{W}^{(1)}$ and for $b_2^{(1)}$:

$$\frac{\partial J}{\partial w_{21}^{(1)}} = \delta_2^{(1)} \cdot x_1 = (-0.020)(0) = 0.000$$
$$\frac{\partial J}{\partial w_{22}^{(1)}} = \delta_2^{(1)} \cdot x_2 = (-0.020)(1) = -0.020$$
$$\frac{\partial J}{\partial b_2^{(1)}} = \delta_2^{(1)} \cdot 1 = -0.020$$

6 Summary of All Gradients

Parameter	Current value	Gradient $\partial J/\partial(\cdot)$
$w_1^{(2)}$	0.5	-0.084
$w_2^{(2)}$	0.5	-0.105
$b^{(2)}$	0.0	-0.168
$w_{11}^{(1)}$	0.5	0.000
$w_{12}^{(1)}$	-0.5	-0.021
$b_1^{(1)}$	0.5	-0.021
$w_{21}^{(1)}$	0.5	0.000
$w_{22}^{(1)}$	0.5	-0.020
$b_2^{(1)}$	0.0	-0.020

All non-zero gradients are **negative**, so gradient descent is expected to move those parameters up. The two weights attached to $x_1 = 0$ ($w_{11}^{(1)}$ and $w_{21}^{(1)}$) have zero gradient and should stay put.

7 Update

With all gradients in hand, we apply one gradient descent step (Section 3.2) to every parameter:

$$w \longleftarrow w - \eta \frac{\partial J}{\partial w}, \qquad \eta = 0.1.$$

Parameter	Old	old $-\ \eta \cdot$ gradient	New
$w_1^{(2)}$	0.5	$0.5 - 0.1(-0.084)$	0.508
$w_2^{(2)}$	0.5	$0.5 - 0.1(-0.105)$	0.510
$b^{(2)}$	0.0	$0.0 - 0.1(-0.168)$	0.017
$w_{11}^{(1)}$	0.5	$0.5 - 0.1(0.000)$	0.500
$w_{12}^{(1)}$	-0.5	$-0.5 - 0.1(-0.021)$	-0.498
$b_1^{(1)}$	0.5	$0.5 - 0.1(-0.021)$	0.502
$w_{21}^{(1)}$	0.5	$0.5 - 0.1(0.000)$	0.500
$w_{22}^{(1)}$	0.5	$0.5 - 0.1(-0.020)$	0.502
$b_2^{(1)}$	0.0	$0.0 - 0.1(-0.020)$	0.002

Running the forward pass again on the same input $(x_1, x_2) = (0, 1)$ with the new param-

eters:

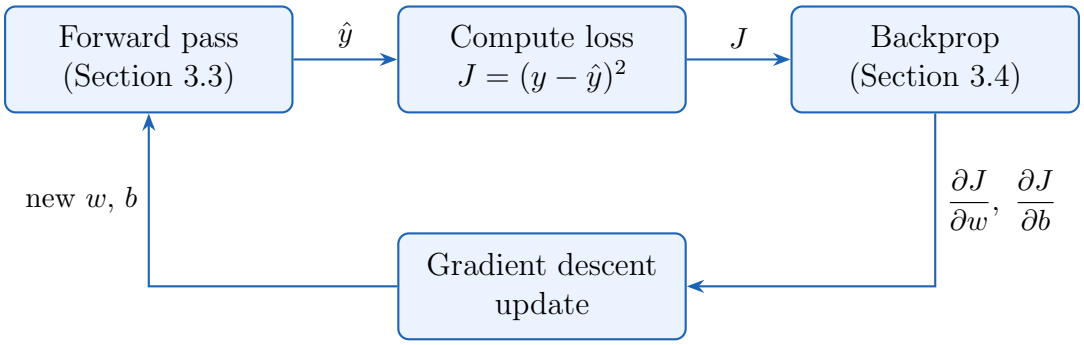
Quantity	Before update	After update
Prediction $\hat{y}$	0.637	0.643
Loss J	0.132	0.127

After one update step the prediction has moved closer to the target $y = 1$, and the loss has dropped.

One forward–backward pass updates weights for one input. In practice we cycle through all four XOR inputs (or use mini-batches), accumulate gradients, and take one update step per batch. After hundreds or thousands of such cycles the network converges and correctly classifies all four XOR inputs.

8 The Big Picture

The full training loop is now complete in principle:



Note: Plain gradient descent works, but it can be slow or sensitive to the learning rate. "Adam" is a smarter update rule that adapts the step size for each weight automatically.